

Esperas en Selenium Java – Front Web

Tipos, combinaciones y excepciones

En Selenium WebDriver existen **3 tipos de esperas** para sincronizar los tests con el comportamiento del Front Web. Una buena estrategia de esperas es clave para evitar **flakiness**, **timeouts innecesarios** y errores intermitentes.

1. Implicit Wait (Espera implícita)

Qué es

Configura un tiempo máximo global para que WebDriver **espere a que los elementos aparezcan en el DOM** antes de lanzar error.

Características

- Aplica solo a `findElement` / `findElements`
- No espera visibilidad ni estado clicable
- Afecta a todo el driver
- Se acumula mal si se mezcla con Explicit Wait

Uso

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

Ventajas

- Simple de usar
- Reduce fallos básicos por carga lenta

Desventajas

- Esperas innecesarias
- Difícil de depurar
- Problemas al combinar con Explicit Wait

Recomendación

👉 Evitarla o usarla en 0–1s. En proyectos modernos se prefiere **Explicit + Fluent**.

2. Explicit Wait (WebDriverWait + ExpectedConditions)

Qué es

Espera **una condición específica** hasta que se cumpla o expire el timeout.

Características

- Espera inteligente y declarativa
- Ideal para interacción con UI
- La más usada y recomendada

Uso base

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

Condiciones más usadas

- `presenceOfElementLocated` → existe en el DOM
 - `visibilityOfElementLocated` → existe y se ve
 - `elementToBeClickable` → visible y habilitado
 - `invisibilityOfElementLocated` → desaparece (spinner/overlay)
 - `textToBePresentInElementLocated`
 - `attributeToBe` / `attributeContains`
 - `urlContains`, `titleContains`
 - `stalenessOf` → elemento viejo por re-render
-

Combinaciones típicas con Explicit Wait

1) Presente → Visible → Click

```
wait.until(ExpectedConditions.presenceOfElementLocated(By.id("save")));  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("save")));  
wait.until(ExpectedConditions.elementToBeClickable(By.id("save"))).click();
```

2) Overlay/Spinner OUT → Click

```
wait.until(ExpectedConditions.invisibilityOfElementLocated(By.cssSelector(".spinner,.overlay  
")));  
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("button.submit"))).click(  
);
```

3) Click → navegación / pantalla nueva

```
wait.until(ExpectedConditions.elementToBeClickable(By.id("login"))).click();  
wait.until(ExpectedConditions.urlContains("/dashboard"));
```

4) Esperar estado por atributo

```
By btn = By.id("save");  
wait.until(ExpectedConditions.attributeToBe(btn, "disabled", "false"));  
wait.until(ExpectedConditions.elementToBeClickable(btn)).click();
```

5) Mensajes, toasts y validaciones

```
By toast = By.cssSelector(".toast");  
wait.until(ExpectedConditions.visibilityOfElementLocated(toast));  
wait.until(ExpectedConditions.textToBePresentInElementLocated(toast, "Guardado"));
```

6) Re-render controlado con staleness

```
WebElement row = driver.findElement(By.cssSelector("tr[data-id='123']"));  
driver.findElement(By.id("refresh")).click();
```

```
wait.until(ExpectedConditions.stalenessOf(row));  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("tr[data-id='123']"))) ;
```

3. Fluent Wait

Qué es

Es una versión avanzada de Explicit Wait que permite:

- Configurar **polling**
- Ignorar **excepciones específicas**
- Manejar escenarios inestables o altamente dinámicos

Uso base

```
Wait<WebDriver> wait = new FluentWait<>(driver)  
    .withTimeout(Duration.ofSeconds(15))  
    .pollingEvery(Duration.ofMillis(200))  
    .ignoring(NoSuchElementException.class)  
    .ignoring(StaleElementReferenceException.class);
```

Combinaciones típicas con Fluent Wait

1) Encontrar y clicar ignorando Stale + NoSuch

By buy = By.cssSelector("button.buy");

```
wait.until(d -> {  
    WebElement b = d.findElement(buy);  
    if (b.isDisplayed() && b.isEnabled()) {  
        b.click();  
        return true;  
    }  
    return false;  
});
```

2) Esperar que desaparezca overlay con polling fino

By overlay = By.cssSelector(".overlay,.spinner");

```
wait.until(d ->  
    d.findElements(overlay).isEmpty() ||  
    !d.findElement(overlay).isDisplayed()  
);
```

3) Esperar valor estable (evitar animaciones)

By total = By.id("total");

```
wait.until(d -> {  
    String a = d.findElement(total).getText().trim();  
    try { Thread.sleep(150); } catch (InterruptedException ignored) {}  
    String b = d.findElement(total).getText().trim();  
    return !a.isEmpty() && a.equals(b);  
});
```

4) Fluent + ExpectedConditions

By el = By.cssSelector("#email");

```
wait.until(d ->  
    ExpectedConditions.visibilityOfElementLocated(el).apply(d)  
);
```

Excepciones comunes en Selenium (Front Web)

Relacionadas con esperas

- **TimeoutException** → la condición no se cumplió a tiempo
✓ Revisar condición, no solo aumentar timeout
-

Relacionadas con elementos / DOM

- **NoSuchElementException** → no existe en ese momento
 - **StaleElementReferenceException** → re-render del DOM
 - **ElementNotInteractableException** → no interactuable
 - **ElementClickInterceptedException** → overlay u otro elemento bloquea
 - **InvalidElementStateException** → estado inválido (readonly, disabled)
-

Relacionadas con contexto / navegación

- **NoSuchFrameException**
 - **NoSuchWindowException**
 - **UnhandledAlertException**
 - **JavaScriptException**
 - **InvalidSelectorException**
 - **SessionNotFoundException**
 - **WebDriverException** (genérica)
-

Reglas prácticas (best practices)

- Preferir **Explicit + Fluent**, evitar Implicit
- Esperar **overlay OUT** antes de interactuar
- Usar **By** en waits (no WebElement cacheado)
- Fluent cuando haya **re-render, polling o flakiness**
- Evitar `Thread.sleep()` salvo casos extremos